

Python Classes and Objects

[< Previous](#)[Next >](#)

Python Classes/Objects

Python is an object oriented programming language.

Almost everything in Python is an object, with its properties and methods.

A Class is like an object constructor, or a "blueprint" for creating objects.

Create a Class

To create a class, use the keyword `class`:

Example

[Get your own Python Server](#)

Create a class named MyClass, with a property named x:

```
class MyClass:  
    x = 5
```

[Try it Yourself »](#)

Create Object

Now we can use the class named MyClass to create objects:

Example

Create an object named p1, and print the value of x:

```
p1 = MyClass()  
print(p1.x)
```

[Try it Yourself »](#)

The __init__() Method

The examples above are classes and objects in their simplest form, and are not really useful in real life applications.

To understand the meaning of classes we have to understand the built-in `__init__()` method.

All classes have a method called `__init__()`, which is always executed when the class is being initiated.

Use the `__init__()` method to assign values to object properties, or other operations that are necessary to do when the object is being created:

Example

Create a class named Person, use the `__init__()`

LEARN P
GET CER
CHECK IT

w³
schools

COLOR
PICKER



method to assign values for name and age:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("John", 36)

print(p1.name)
print(p1.age)
```

Try it Yourself »

Note: The `__init__()` method is called automatically every time the class is being used to create a new object.

ADVERTISEMENT

[REMOVE ADS](#)

The `__str__()` Method

The `__str__()` method controls what should be returned when the class object is represented as a string.

If the `__str__()` method is not set, the string representation of the object is returned:

Example

The string representation of an object WITHOUT the `__str__()` method:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("John", 36)

print(p1)
```

Try it Yourself »

Example

The string representation of an object WITH the `__str__()` method:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}({self.age})"

p1 = Person("John", 36)

print(p1)
```

Try it Yourself »

Create Methods

You can create your own methods inside objects. Methods in objects are functions that belong to the object.

Let us create a method in the Person class:

Example

Insert a function that prints a greeting, and execute it on the p1 object:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " +
              self.name)

p1 = Person("John", 36)
p1.myfunc()
```

Try it Yourself »

Note: The **self** parameter is a reference to the current instance of the class, and is used to access variables that belong to the class.

The self Parameter

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Certificates ▼](#)[In](#)[HTML](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[HOW TO](#)[W3.CSS](#)[C](#)[C](#)[Python Booleans](#)[Python Operators](#)[Python Lists ▼](#)[Python Tuples ▼](#)[Python Sets ▼](#)[Python Dictionaries](#)

It does not have to be named **self**, you can call it whatever you like, but it has to be the first parameter of any function in the class:

Example

Python If...Else
Python Match
Python While Loops
Python For Loops
Python Functions
Python Decorators
Python Range
Python Lambda
Python Arrays
Python OOP
Python Classes/Objects
Python Inheritance
Python Iterators
Python Polymorphism
Python Scope
Python Modules
Python Dates
Python Math
Python JSON
Python RegEx
Python PIP
Python Try...Except
Python String Formatting
Python None
Python User Input
Python VirtualEnv

File Handling

Use the words *mysillyobject* and *abc* instead of *self*:

```
class Person:
    def __init__(mysillyobject, name, age):
        mysillyobject.name = name
        mysillyobject.age = age

    def myfunc(abc):
        print("Hello my name is " + abc.name)

p1 = Person("John", 36)
p1.myfunc()
```

[Try it Yourself »](#)

Modify Object Properties

You can modify properties on objects like this:

Example

Set the age of p1 to 40:

```
p1.age = 40
```

[Try it Yourself »](#)

Delete Object Properties

You can delete properties on objects by using the [del](#) keyword:

Example

Delete the age property from the p1 object:

```
del p1.age
```

Try it Yourself »

Delete Objects

You can delete objects by using the `del` keyword:

Example

Delete the p1 object:

```
del p1
```

Try it Yourself »

The pass Statement

`class` definitions cannot be empty, but if you for some reason have a `class` definition with no content, put in the `pass` statement to avoid getting an error.

Example

```
class Person:  
    pass
```

Try it Yourself »